Test Cases

Function 1: Register

Parameters:

- String – first_name
- String – last_name
- String – email
- String – password
- String – role
- String – phone (optional)

Output data type: HTTP Response (201 / 400 / 409)

Description: Register a new user with validation for required fields, email format, role validation, and duplicate email check.

Input parameter value combinations:

1. null, "john", "test@example.com", "pass123", "customer", "" -> 400 (first_name is required)
2. "john", null, "test@example.com", "pass123", "customer", "" -> 400 (last_name is required)
3. "john", "doe", null, "pass123", "customer", "" -> 400 (email is required)
4. "john", "doe", "test@example.com", null, "customer", "" -> 400 (password is required)
5. "john", "doe", "test@example.com", "pass123", null, "" -> 400 (role is required)
6. "john", "doe", "invalid-email", "pass123", "customer", "" -> 400 (Invalid email address)
7. "john", "doe", "test@example.com", "123", "customer", "" -> 400 (Password must be at least 4 characters)
8. "john", "doe", "test@example.com", "pass123", "invalid_role", "" -> 400 (Invalid role)
9. "john", "doe", "existing@example.com", "pass123", "customer", "" -> 400 (Email already registered)
10. "john", "doe", "test@example.com", "pass123", "customer", "" -> 201 (User registered successfully)
11. "jane", "smith", "jane@example.com", "pass123", "master", "1234567890" -> 201 (User registered successfully)
12. "admin", "user", "admin@example.com", "pass123", "operator", "" -> 201 (User registered successfully)

```
[Fact]
    public void Register()
    {
        Assert.Equal(400, Register(null, "john", "test@example.com", "pass123", "customer", ""));
        Assert.Equal(400, Register("john", null, "test@example.com", "pass123", "customer", ""));
        Assert.Equal(400, Register("john", "doe", null, "pass123", "customer", ""));
        Assert.Equal(400, Register("john", "doe", "test@example.com", null, "customer", ""));
        Assert.Equal(400, Register("john", "doe", "test@example.com", "pass123", null, ""));
        Assert.Equal(400, Register("john", "doe", "invalid-email", "pass123", "customer", ""));
        Assert.Equal(400, Register("john", "doe", "test@example.com", "123", "customer", ""));
        Assert.Equal(400, Register("john", "doe", "test@example.com", "pass123", "invalid_role",
""));
        Assert.Equal(400, Register("john", "doe", "existing@example.com", "pass123", "customer",
""));
        Assert.Equal(201, Register("john", "doe", "test@example.com", "pass123", "customer", ""));
        Assert.Equal(201, Register("jane", "smith", "jane@example.com", "pass123", "master",
"1234567890"));
        Assert.Equal(201, Register("admin", "user", "admin@example.com", "pass123", "operator",
""));
    }
```

Function 2: Create Request

Parameters:

- Integer – customer_id
- Integer – service_id
- String – address
- String – description (optional)
- DateTime – scheduled_at (optional)

Output data type: HTTP Response (201 / 400 / 404)

Description: Create a new repair request with validation for required fields and entity existence.

Input parameter value combinations:

1. null, 1, "123 Main St", "Need repair", null -> 400 (Not logged in)

2. 0, 1, "123 Main St", "Need repair", null -> 400 (Not logged in)
3. 1, null, "123 Main St", "Need repair", null -> 400 (Please select a service)
4. 1, 0, "123 Main St", "Need repair", null -> 400 (Please select a service)
5. 1, 1, null, "Need repair", null -> 400 (Address is required)
6. 1, 1, "", "Need repair", null -> 400 (Address is required)
7. 999, 1, "123 Main St", "Need repair", null -> 404 (Customer not found)
8. 1, 999, "123 Main St", "Need repair", null -> 404 (Service not found)
9. 1, 1, "x" * 256, "Need repair", null -> 400 (Address is too long)
10. 1, 1, "123 Main St", "x" * 501, null -> 400 (Description is too long)
11. 1, 1, "123 Main St", "Need repair", null -> 201 (Request created successfully)
12. 1, 1, "456 Oak Ave", "", null -> 201 (Request created with empty description)

Function 3: Get Requests

Parameters:

- String – role
- Integer – user_id

Output data type: HTTP Response (200 / 400)

Input parameter value combinations:

1. null, 1 -> 400 (role and user_id are required)
2. "customer", null -> 400 (role and user_id are required)
3. null, null -> 400 (role and user_id are required)
4. "", 1 -> 400 (role and user_id are required)
5. "customer", 1 -> 200 (Returns customer's requests only)
6. "operator", 1 -> 200 (Returns all requests)
7. "master", 1 -> 200 (Returns all requests)